



UNIVERSITY OF MINNESOTA

Driven to Discover®

Collision Detection

CSCI 5980: Game Engine Architecture

Evan Suma Rosenberg | CSCI 5980 | Spring 2026

This course content is offered under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International license.



Collision and Physics

- In the real world, solid objects don't pass through each other; in a game world, objects do nothing unless we tell them to
- The **collision detection system** ensures objects do not interpenetrate
- A collision system is often closely integrated with a **physics engine**
- "Physics" in games really means **rigid body dynamics simulation**
 - A rigid body is an idealized, infinitely hard, non-deformable solid object
 - Dynamics: how these bodies move and interact under the influence of forces

Why Physics?

- Dynamics simulations produce highly interactive, naturally chaotic motion that is hard to achieve with canned animation clips
- The dynamics simulation depends heavily on collision detection for behaviors like bouncing, sliding, rolling, and coming to rest
- A collision system can also be used **standalone** without dynamics
 - All games with moving objects in 2D or 3D need some form of collision detection

Things You Can Do with a Physics System

- Detect collisions between dynamic objects and static world geometry
- Simulate free rigid bodies under gravity and other forces
- Spring-mass systems
- Destructible buildings and structures
- Ray and shape casts (line of sight, bullet impacts, etc.)

Things You Can Do with a Physics System

- Trigger volumes (detect when objects enter/leave predefined regions)
- Complex machines (cranes, moving platform puzzles)
- Traps (e.g., an avalanche of boulders)
- Drivable vehicles with realistic suspensions
- Rag doll character deaths
- Powered rag doll: blending traditional animation and rag doll physics

Things You Can Do with a Physics System

- Dangling props, semi-realistic hair, clothing
- Cloth simulations
- Water surface simulations and buoyancy
- Audio propagation
- Physics can also run **offline** as a preprocessing step to generate animation clips (e.g., NaturalMotion's Endorphin)

Physics and Game Genres

- **Simulations:** Flight Simulator, Gran Turismo.
 - Realism is the goal, so rigid body dynamics fits extremely well
- **Physics puzzle games:** Bridge Builder, The Incredible Machine, Crayon Physics.
 - Physics **is** the core mechanic
- **Sandbox games:** Besiege, Spore, LittleBigPlanet, Minecraft.
 - Physics can be fun in itself, but realism is often traded for increased fun factor
- **Goal-based and story-driven games**
 - Giving up control in exchange for simulation can inhibit accomplishing goals or telling the story.
 - Physics must be applied **judiciously**

Impact of Physics: Design

- **Predictability**

- The chaotic variability of physics is a source of unpredictability.
- If something must happen a certain way every time, animate it instead.

- **Tuning and control:**

- Physics parameters (gravity, restitution) offer some control.
- Results are indirect and hard to visualize.

- **Emergent behaviors**

- Physics sometimes introduces unexpected features
- (e.g., rocket-launcher jumping in Team Fortress Classic, flying Warthog in Halo)

- In general, game design should drive the physics requirements, not the other way around

Impact of Physics: Engineering

- **Tools pipeline**

- A good collision/physics pipeline takes time to build and maintain.

- **User interface**

- How does the player interact with physics objects?
- (e.g., shoot them, pick them up, gravity gun, etc.)

- **Collision models**

- Physics-driven collision may need more detail than non-physics collision.

Impact of Physics: Engineering

- **AI**
 - Pathing may not be predictable.
 - Dynamic cover points can move or blow up.
- **Misbehaved objects**
 - Physics-driven objects may bounce unexpectedly or jitter badly
 - Collision filtering and sleep mechanisms are needed.
- **Rag doll physics**
 - Requires fine-tuning
 - Suffers from instability when animations drive body parts into penetration

Impact of Physics: Engineering

- **Graphics**

- Physics-driven motion can affect bounding volumes.
- Destructible objects can invalidate precomputed lighting.

- **Networking**

- Non-gameplay physics can be simulated independently on each client.
- Gameplay-affecting physics must be simulated on the server and replicated.

- **Record and playback**

- Requires deterministic behavior across all engine systems.
- A non-deterministic physics simulation can make this very difficult.

Impact of Physics: Art and Production

- **Art impacts**
 - Additional tool and workflow complexity (rigging objects with mass, friction, constraints).
 - May need multiple versions of objects (pristine vs. destructible).
 - Loss of artistic control due to unpredictable physics-driven motion.
- **Other impacts**
 - Requires close cooperation between engineering, art, audio, and design.
 - Adds to development costs, technical complexity, and risk.
- **Despite these impacts, most teams today choose to integrate rigid body dynamics.**

Collision/Physics Middleware

Collision/Physics Middleware

- **ODE** (Open Dynamics Engine)
 - Open source, free, full source code available.
- **Bullet**
 - Open source, used in games and film.
 - Supports continuous collision detection (CCD/TOI).
 - Collision engine can be used standalone.
- **PAL** (Physics Abstraction Layer)
 - Open source library allowing multiple physics SDKs on a single project.

Collision/Physics Middleware

- **PhysX**

- Originally Novodex by Ageia, acquired by NVIDIA.
- Can run on GPU or CPU. Free CPU version; paid source code access.
- Combined with APEX scalable dynamics framework.

- **Havok**

- Gold standard in commercial physics.
- Richest feature set, excellent performance.
- Add-on products include vehicle physics, destructible environments, animation SDK with rag doll integration.
- Most expensive option

The Collision Detection System

The Collision Detection System

- The primary purpose: determine whether any objects in the game world have come into contact
- Each logical object is represented by one or more **geometric shapes**
 - Usually simple: spheres, boxes, capsules
- The collision system is essentially a **glorified geometric intersection tester**
- Beyond yes/no intersection, the system provides **contact information** used to:
 - Separate interpenetrating objects before rendering
 - Provide support so objects come to rest
 - Trigger gameplay events (missile explosions, health boosts)
 - Drive physics behaviors (bouncing, rolling, sliding)

Collidable Entities

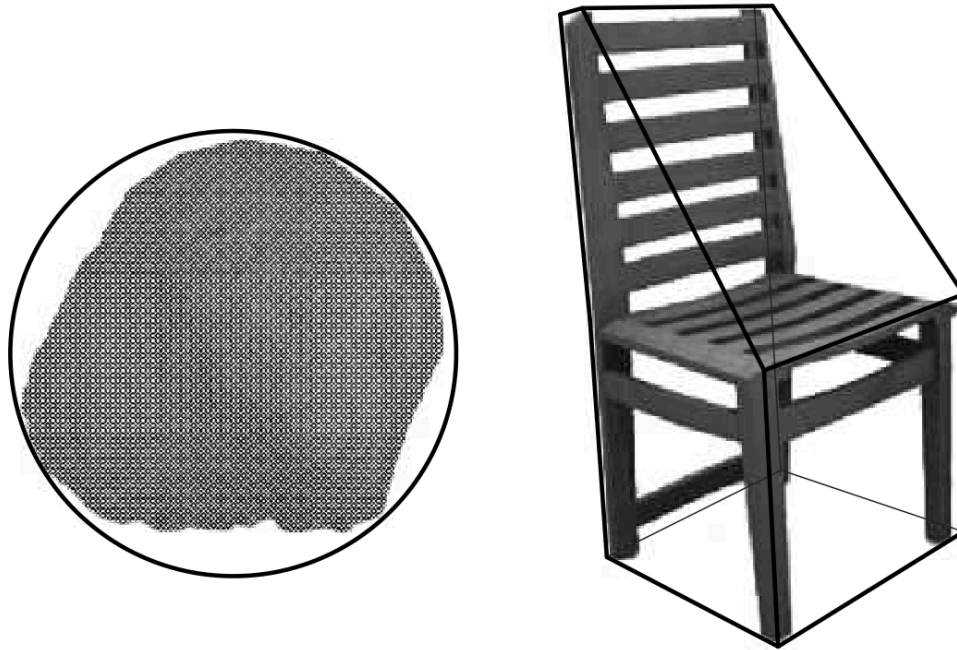


Figure 13.1. Simple geometric shapes are often used to approximate the collision volumes of the objects in a game.

Collidable Entities

- To collide, an object needs a **collision representation**, separate from its gameplay and visual representations
- We favor shapes that are **geometrically and mathematically simple**
 - Rock modeled as a sphere, car hood as a box, human body as interconnected capsules
- A collidable entity contains two basic pieces of information:
 - **Shape**: Describes geometric form (sphere, box, capsule, etc.)
 - **Transform**: Describes position and orientation in the game world
- Havok: `hkpCollidable` ; PhysX: `NxActor`

Why Collidables Need Transforms

- Shapes are usually defined at the **origin** in a canonical orientation.
 - A transform positions and orients them in world space.
- Moving a complex shape could be expensive if we moved every feature individually.
 - A transform makes **any shape** movable inexpensively.
- Shape data can take up nontrivial memory, so **multiple collidables can share a single shape**
 - e.g., all cars in a racing game share one car shape.
- A game object may have **zero** collidables (no collision needed), **one** (simple rigid body), or **multiple** (articulated objects like a robot arm)

The Collision/Physics World

- A collision system keeps track of all collidables via a singleton data structure: the **collision world**
 - Havok: `hkpWorld` , PhysX: `NxScene` , ODE: `dSpace`
- Advantages of a private data structure:
 - Contains **only collidable objects**, eliminating irrelevant data
 - Data can be organized for maximum **cache coherency**
 - Effective **encapsulation** for understandability, maintainability, and reuse
- When a game has dynamics, the physics system **shares** the world data structure
 - The physics system drives the collision system, running tests at least once per simulation step
 - Often called the **collision/physics world** or just the **physics world**

Shape Concepts

- A shape is a **region of space** described by a boundary, with a definite inside and outside
- In 3D, a shape has volume, and its boundary is a curved surface or a polyhedron
- Some objects (terrain, thin walls) are best represented by **surfaces** (2D entities with front/back but no inside/outside)
- Most collision SDKs support both closed volumes and open surfaces under the term "shape"
- Surfaces can be given volume via an optional **extrusion parameter** (thickness) to reduce missed collisions with small, fast objects

Intersection and Contact

- **Intersection:** The set of all points that lie inside both shapes (from set theory)
 - In games, we typically want to know **whether** two objects intersect, not the exact set of intersection points.
- **Contact information** is packaged into a data structure for each detected contact. Typically includes:
 - A **separating vector** to efficiently move objects apart
 - Which two collidables (and which specific shapes/features) were in contact
 - Possibly the velocity of bodies projected onto the separating normal

Convexity

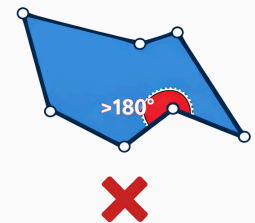
- **Convex** shape:
 - No ray originating inside passes through its surface more than once
 - All interior angles are less than 180 degrees
- It is **simpler and cheaper** to detect intersections between convex shapes than concave ones

Convex Polygon



All interior angles less than 180°

Non-Convex Polygon

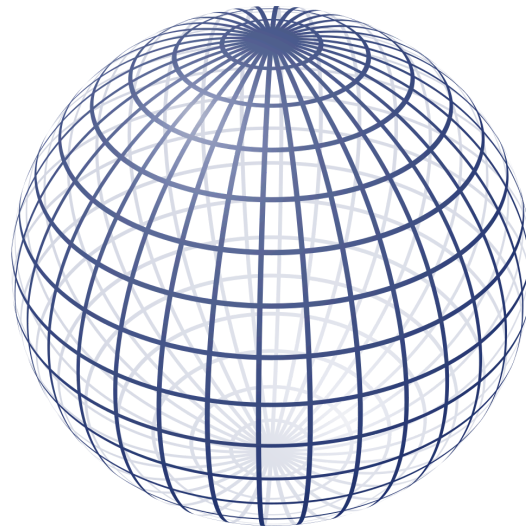


One reflex angle greater than 180°

Collision Primitives

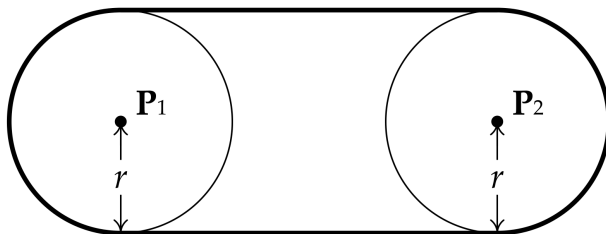
Spheres

- The simplest and most efficient 3D collision primitive
- Represented by a **center point** and a **radius**
- Can be packed into a four-element floating-point vector
 - Works well with **SIMD** (Single Instruction, Multiple Data) math libraries



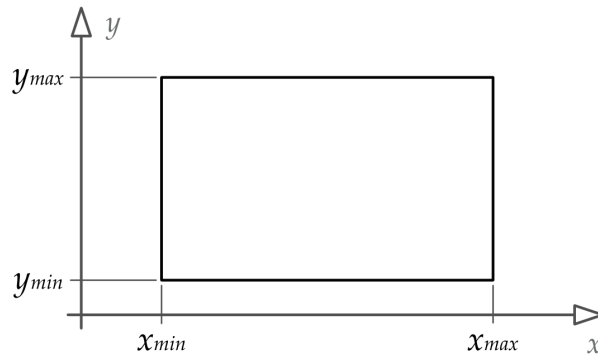
Capsules

- A pill-shaped volume: a cylinder with two hemispherical end caps
- Can be thought of as a **swept sphere** (shape traced as a sphere moves from point A to point B)
- Represented by **two points and a radius**
- More efficient to intersect than cylinders or boxes
- Often used for roughly cylindrical objects like human body limbs



Axis-Aligned Bounding Boxes (AABBs)

- A rectangular volume (cuboid) whose faces are **parallel to the coordinate axes**
- Defined by two points: **(x_min, y_min, z_min)** and **(x_max, y_max, z_max)**
- Primary benefit: Can be tested for interpenetration **very efficiently**
- Limitation:
 - Must remain axis-aligned.
 - If the object rotates, the AABB must be recalculated.
 - May become a poor approximation of the shape.



AABB in two-dimensions

Axis-Aligned Bounding Boxes (AABBs)

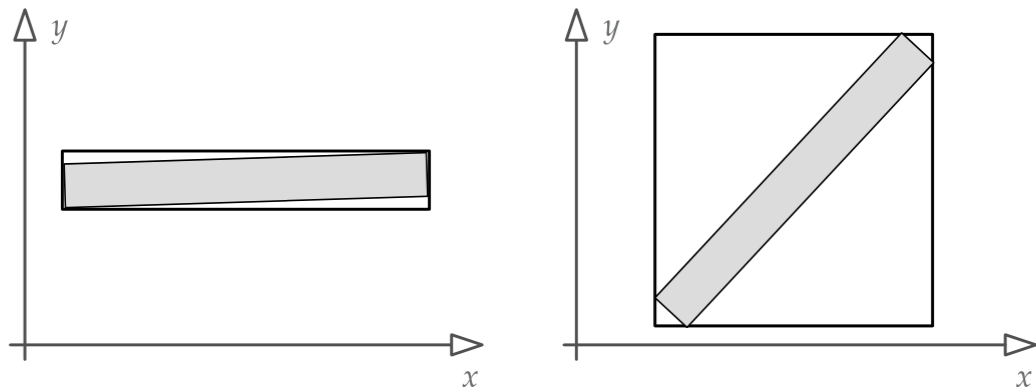


Figure 13.4. An AABB is only a good approximation to a box-shaped object when the object's principal axes are roughly aligned with the coordinated system's axes.

Oriented Bounding Boxes (OBBs)

- An axis-aligned box permitted to **rotate** relative to its coordinate system
- Represented by three **half-dimensions** and a **transformation**
- Better at fitting arbitrarily oriented objects than AABBs
- Representation is still quite simple

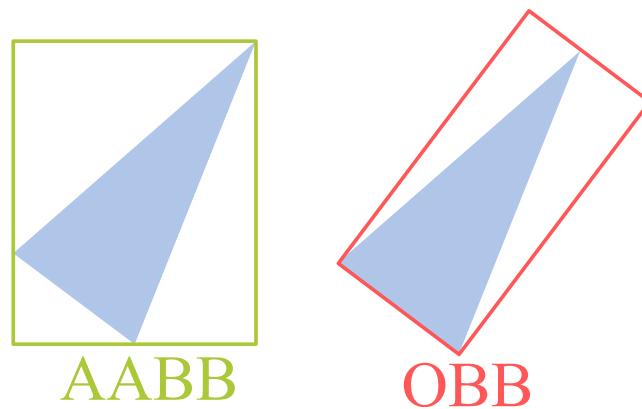


Image Source

Discrete Oriented Polytopes (DOPs)

- A more general case of the AABB and OBB
- A convex polytope constructed by sliding planes along their normals until they contact the object
- An AABB is a **6-DOP** with plane normals parallel to coordinate axes
- An OBB is also a **6-DOP** with plane normals parallel to the object's principal axes
- A **k-DOP** uses an arbitrary number of planes k

Discrete Oriented Polytopes (DOPs)

- Common construction: start with an OBB, then bevel edges/corners at 45 degrees for a tighter fit (e.g., a 14-DOP)

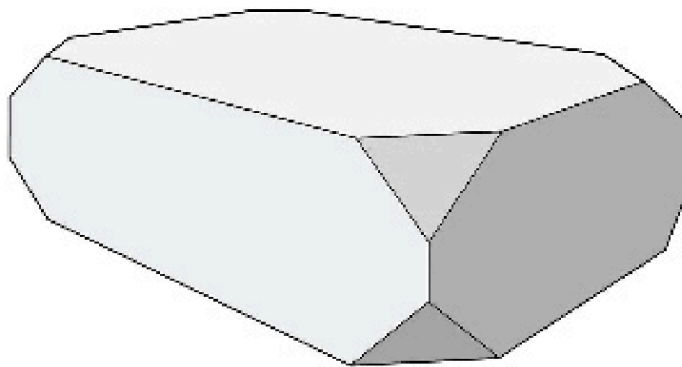


Figure 13.5. An OBB that has been beveled on all eight corners is known as a 14-DOP.

Arbitrary Convex Volumes

- Most collision engines allow a 3D artist to construct arbitrary convex volumes in a package like Maya
- An offline tool verifies the shape is a convex polyhedron, then converts triangles into a collection of planes (essentially a k-DOP)
- If found to be non-convex, it can be represented as a **polygon soup** instead
- More expensive to intersection-test than simpler primitives
- However, efficient algorithms like **GJK** apply specifically because they are convex

Poly Soup

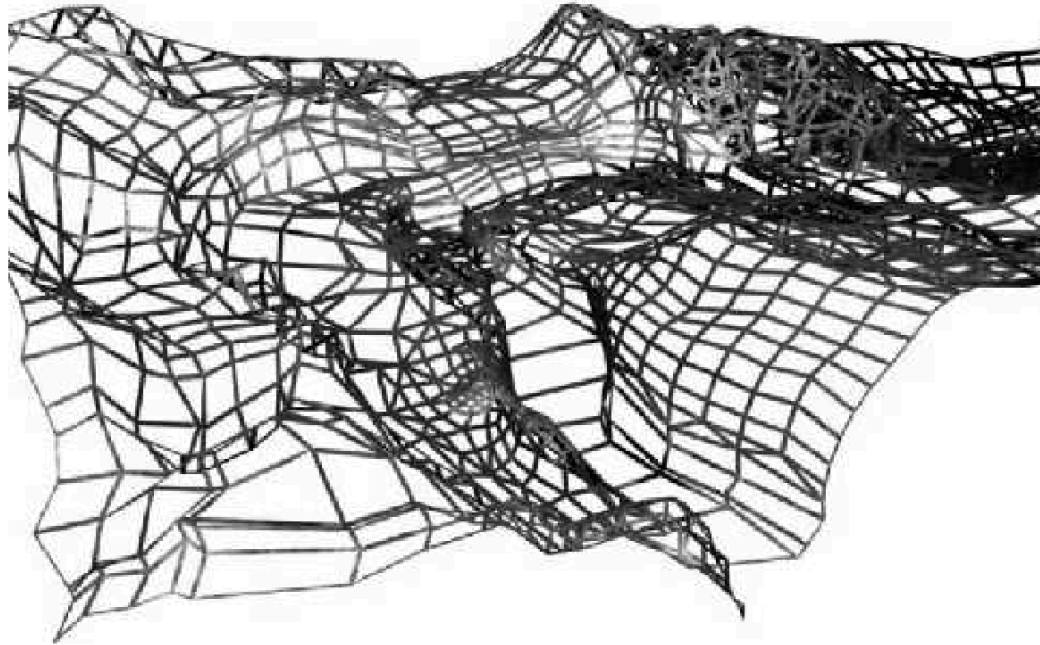


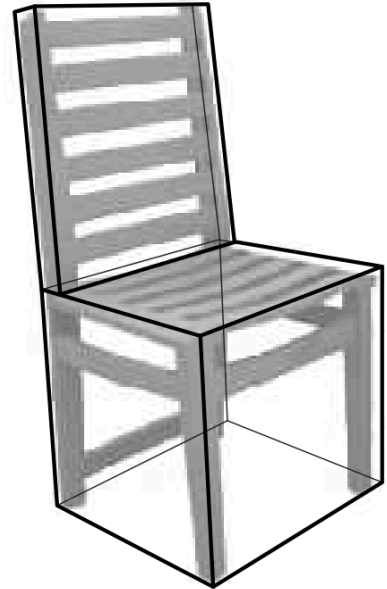
Figure 13.7. A poly soup is often used to model complex static surfaces such as terrain or buildings.

Poly Soup

- Totally arbitrary, non-convex shapes constructed from triangles or other polygons
- Often used for complex static geometry like **terrain and buildings**
- **Most expensive** kind of collision test: must test every individual triangle and handle spurious intersections at shared edges
- Most games limit poly soup to objects **not in the dynamics simulation**
- A poly soup may not have a clear "inside" vs. "outside"
 - Consistent vertex winding orders give a notion of "front" and "back"
 - Combined with open/closed metadata, this can be used to determine inside/outside

Compound Shapes

- Objects that cannot be approximated by a single shape can use a **collection of shapes**
 - Example: A chair modeled with two boxes (one for the back, one for seat + legs)
- Often more efficient than poly soup for non-convex objects
- Collision systems can test the **convex bounding volume** of the compound shape first
 - Havok calls this **midphase collision detection**
 - If bounding volumes don't intersect, no need to test subshapes



Collision Testing

Collision Testing and Analytical Geometry

- Collision systems use **analytical geometry** (mathematical descriptions of 3D volumes and surfaces) to detect intersections computationally
- For each pair of shape types, a specialized intersection test is needed
- The number of shape-shape combinations is **$O(N^2)$** for N shape types
- This is why collision engines try to **limit the number of primitive types**

Point vs. Sphere

Form the separation vector **s** between point **p** and sphere center **c**, then check its length against the radius **r**:

$$\mathbf{s} = \mathbf{c} - \mathbf{p}$$

if $|\mathbf{s}| \leq r$, then **p** is inside

Sphere vs. Sphere

Form the vector **s** connecting the two centers, compare its length to the sum of radii:

$$\mathbf{s} = \mathbf{c}_1 - \mathbf{c}_2$$

if $|\mathbf{s}| \leq (r_1 + r_2)$, then spheres intersect

To avoid the square root, square both sides:

$$|\mathbf{s}|^2 = \mathbf{s} \cdot \mathbf{s}$$

if $|\mathbf{s}|^2 \leq (r_1 + r_2)^2$, then spheres intersect

The Separating Axis Theorem

- If an axis can be found along which the **projections** of two convex shapes do not overlap, then the shapes do **not intersect**
- If no such axis exists and the shapes are convex, then they **do intersect**
 - For concave shapes, the absence of a separating axis does not guarantee intersection
- The projection of a convex shape onto an axis is always a **line segment** $c_{\min}$, $c_{\max}$
- When a separating line/plane exists, projections are **disjoint** on the separating axis
- In 3D, the separating line becomes a **separating plane**
 - The separating axis is still an axis

The Separating Axis Theorem

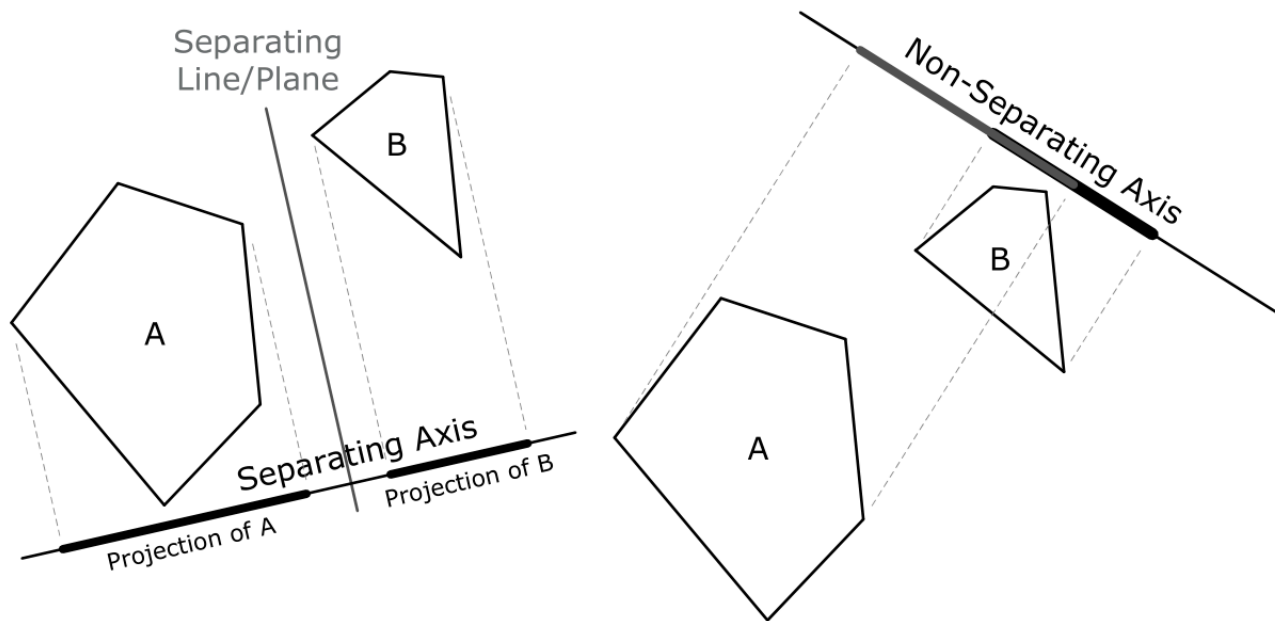


Figure 13.10. The projections of two shapes onto a separating axis are always two disjoint line segments. The projections of these same shapes onto a non-separating axis are not necessarily disjoint. If no separating axis exists, the shapes intersect.

Separating Axis Theorem: Checking Intervals

- For shapes with known potential separating axes, project both shapes onto each candidate axis
- Check whether projection intervals $c^A_{\min}$, $c^A_{\max}$ and $c^B_{\min}$, $c^B_{\max}$ are **disjoint**:
 - Disjoint if $c^A_{\max} < c^B_{\min}$ **or** $c^B_{\max} < c^A_{\min}$
- If any axis yields disjoint intervals, the shapes **do not intersect**
- Sphere-vs-sphere example:
 - The axis parallel to the center-to-center line segment is always a valid separating axis (when the spheres don't intersect).

The Separating Axis Theorem

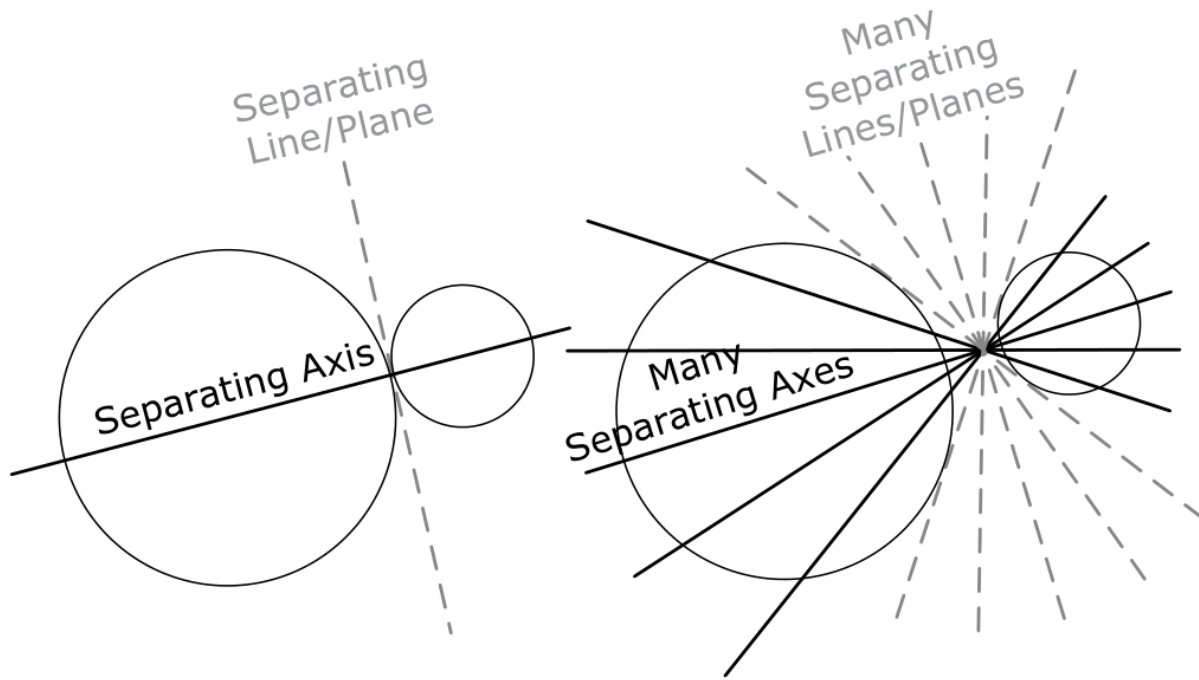


Figure 13.11. When two spheres are an infinitesimal distance apart, the only separating axis lies parallel to the line segment formed by the two spheres' center points.

AABB vs. AABB

- Apply the separating axis theorem with the **three coordinate axes** as candidates
- For each axis, compare the min/max intervals of the two boxes:
 - x-axis: $x^A_{\min}$, $x^A_{\max}$ vs. $x^B_{\min}$, $x^B_{\max}$
 - y-axis and z-axis: similar intervals
- If the intervals overlap along **all three axes**, the AABBs are intersecting
- If any axis has disjoint intervals, they are **not** intersecting

The GJK Algorithm

- An efficient algorithm for detecting intersections between **arbitrary convex polytopes**
- Named after Gilbert, Johnson, and Keerthi (University of Michigan)
- Relies on the **Minkowski difference**
 - Subtract every point in shape B from every point in shape A
- The Minkowski difference of two convex shapes **contains the origin if and only if the shapes intersect**
- The Minkowski difference of two convex shapes is itself **convex**

Minkowski Difference

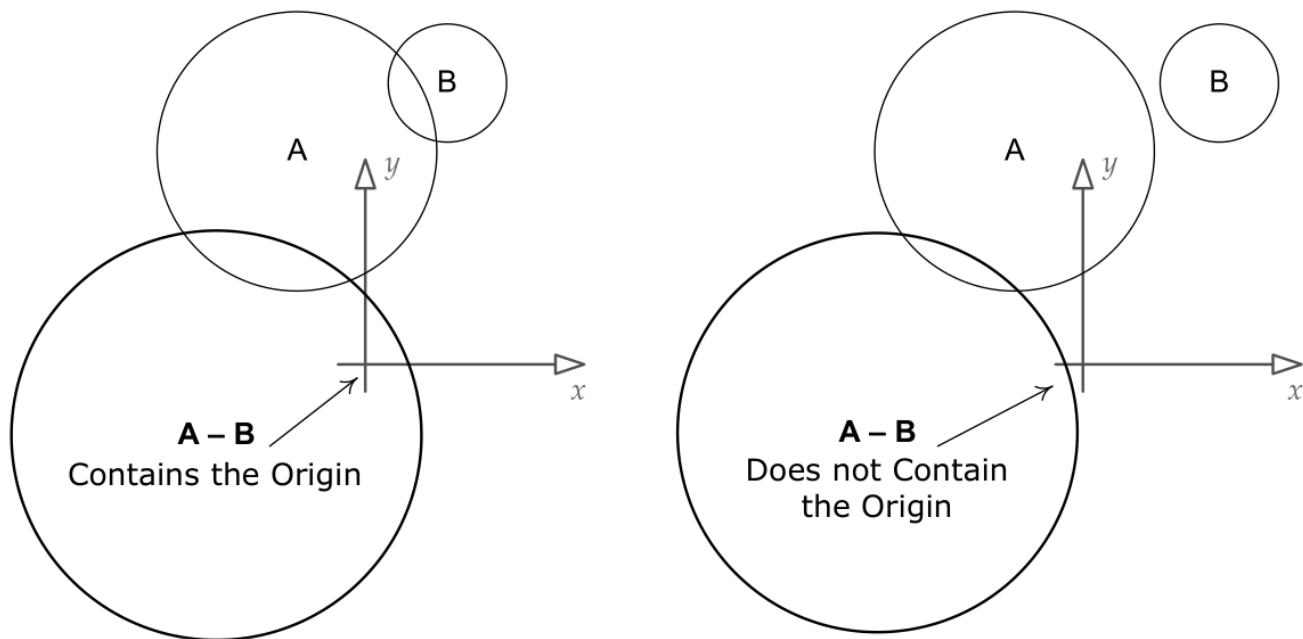


Figure 13.13. The Minkowski difference of two intersecting convex shapes contains the origin, but the Minkowski difference of two non-intersecting shapes does not.

GJK: How It Works

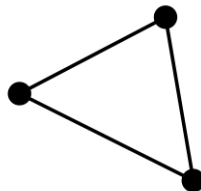
- Goal: find a **tetrahedron** on the convex hull of the Minkowski difference that encloses the origin
- A tetrahedron is a four-point **simplex** (1 point = point, 2 = line, 3 = triangle, 4 = tetrahedron)



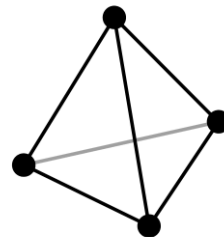
Point



Line Segment



Triangle



Tetrahedron

GJK is Iterative

1. Start with a one-point simplex in the Minkowski difference hull
2. Determine which direction the origin lies relative to the current simplex
3. Find a **supporting vertex** of the Minkowski difference in that direction
4. Add it to the simplex, creating a higher-order simplex
5. If the simplex now contains the origin, the shapes **intersect**
6. If no supporting vertex can bring the simplex closer to the origin, the shapes **do not intersect**

Why GJK Matters

- Handles collision tests between **all convex shapes** in one algorithm!
- The only thing that differs per shape type is the **support function**
- This dramatically reduces number of specialized intersection test cases

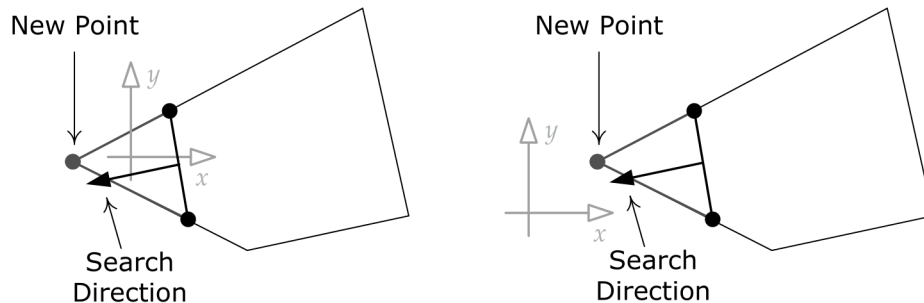


Figure 13.15. In the GJK algorithm, if adding a point to the current simplex creates a shape that contains the origin, we know the shapes intersect; if there is no supporting vertex that will bring the simplex any closer to the origin, then the shapes do not intersect.

Dispatching Intersection Tests

- With N shape types, the pairwise test count is $O(N^2)$
- Many engines use **double dispatch** to select the right test function given two arbitrary shapes
 - A 2D function look-up table keyed by both object types
 - Or nested virtual function calls
- Havok uses **collision agents** (derived from `hkpCollisionAgent`) for each test case
 - Examples: `hkpSphereSphereAgent`, `hkpSphereCapsuleAgent`,
`hkpGskConvexConvexAgent`
 - Managed by `hkpCollisionDispatcher`, a 2D dispatch table

Detecting Collisions between Moving Bodies

- Motion in games is simulated in **discrete time steps**
- Simple approach:
 - Treat each time step as a static snapshot and run static intersection tests
 - Works well when objects aren't moving too fast relative to their sizes
 - Used by default in many engines, including Havok

Static Intersection Tests

- Problem with **small, fast-moving objects**: an object can move farther than its own size between steps, creating a gap in its motion path
 - Objects in the gap will be missed entirely
 - Known as the **bullet through paper** problem or **tunneling**

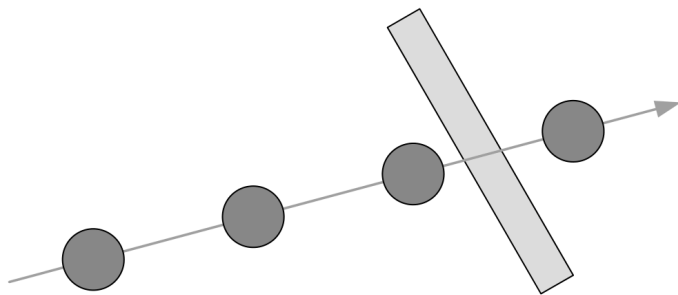


Figure 13.16. A small, fast-moving object can leave gaps in its motion path between consecutive snapshots of the collision world, meaning that collisions might be missed entirely.

Swept Shapes

- Avoid tunneling by testing **swept shapes** formed by moving shapes from previous to current positions
- This amounts to **linearly interpolating** the motion between snapshots

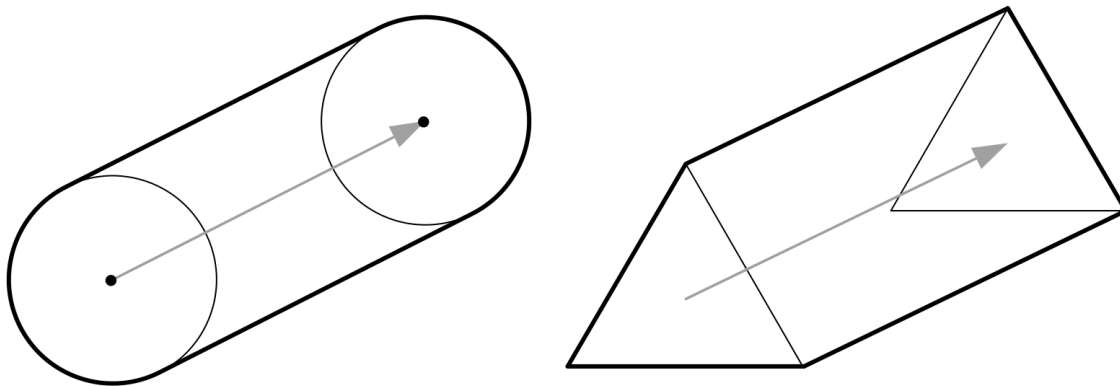


Figure 13.17. A swept sphere is a capsule; a swept triangle is a triangular prism.

Limitations of Swept Shapes

- Linear interpolation is a poor approximation for curved paths
- A rotating convex shape swept along a line segment is **not necessarily convex**
- Swept shape intersection testing is much more complex and expensive

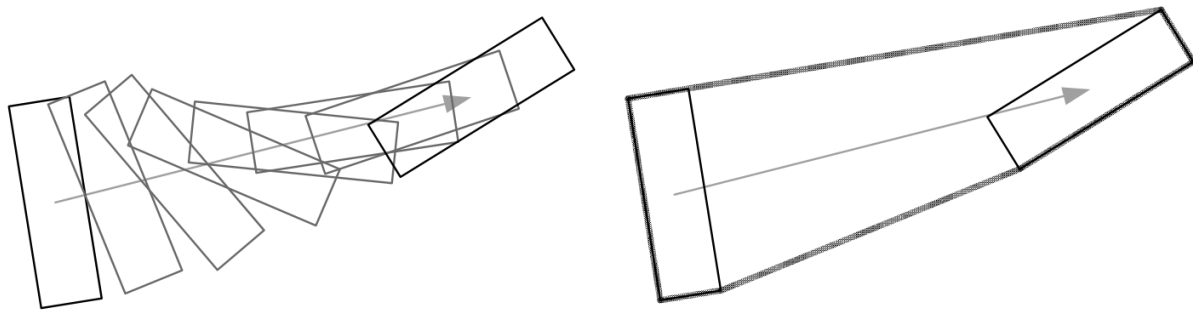


Figure 13.18. A rotating object swept along a line segment does not necessarily generate a convex shape (left). A linear interpolation of the motion does form a convex shape (right), but it can be a fairly inaccurate approximation of what actually happened during the time step.

Continuous Collision Detection (CCD)

- Goal: find the **earliest time of impact (TOI)** between two moving objects over a time interval
- CCD algorithms are generally **iterative**:
 - Maintain position and orientation at both the previous and current time step
 - Linearly interpolate to approximate the collidable's transform at any intermediate time
 - Search for the earliest TOI along the motion path
- Common search algorithms:
 - Brian Mirtich's **conservative advancement** method
 - **Ray cast on the Minkowski sum**
 - Minimum TOI of individual feature pairs

Performance Optimizations

Performance Optimizations

- Collision detection is CPU-intensive for two reasons:
 1. Intersection calculations between two shapes are **nontrivial**
 2. The number of tests grows rapidly with the number of objects
- Brute-force approach
 - Test every pair of objects, yielding **$O(n^2)$** complexity
- In practice, engines use **spatial hashing**, **spatial subdivision**, or **hierarchical bounding volumes** to reduce the test count

Temporal Coherency

- Also called **frame-to-frame coherency**
- When collidables move at reasonable speeds, their positions and orientations are similar from step to step
- Results from previous time steps can be **cached** and reused
- Example:
 - Havok's collision agents (`hkpCollisionAgent`) persist between frames
 - Reuses calculations as long as motion hasn't invalidated them

Spatial Partitioning

- Divide space into smaller regions to greatly reduce pairwise tests
- If two collidables are in **different regions**, no detailed test is needed
- Common hierarchical schemes:
 - **Octrees**
 - **BSP trees** (binary space partitioning)
 - **kd-trees**
 - **Sphere trees**

Broad Phase, Midphase, and Narrow Phase

- **Broad phase**

- Gross AABB tests to find potentially intersecting collidables

- **Midphase**

- Test the coarse bounding volumes of compound shapes.
- Traverse the bounding volume hierarchy to find potentially intersecting subshapes

- **Narrow phase**

- Test individual primitives for actual intersection

Broad Phase: Sweep and Prune

- In all of the major collision/physics engines, broad phase collision detection employs the **sweep and prune** algorithm
- Sort AABB min/max coordinates along each axis, then traverse the sorted lists to find overlaps
- Frame coherency reduces the sort from $O(n \log n)$ to expected $O(n)$

Collision Queries

Collision Queries

- The collision system can also answer **hypothetical questions** about collision volumes:
 - What is the first target a bullet will hit?
 - Can a vehicle move from A to B without hitting anything?
 - Find all enemies within a given radius of a character
- In general, such operations are known as **collision queries**.

Collision Casts

- A **collision cast** determines what a hypothetical object would hit if placed in the world and moved along a ray or line segment
 - Also known as a **trace** or **probe**
- Casts are different from regular collision
 - The cast entity is **not really in the world** and cannot affect other objects

Ray Casting

- The simplest collision cast
- Really a **directed line segment** with start point **p0** and end point **p1**
- Described parametrically:
 - $p(t) = p0 + t*d$, where t ranges from 0 to 1
 - Contact points are returned as **t values**
- Most systems return the **earliest contact** (smallest t)
 - Some can return all contacts

Applications of Ray Casts

- **Line of sight:** Cast from character A's eyes to character B's chest
- **Weapon systems:** Determine bullet hits
- **Player mechanics:** Check for solid ground beneath the character's feet
- **AI systems:** Line of sight checks, targeting, movement queries
- **Vehicle systems:** Locate and snap tires to the terrain

Shape Casting

- How far can an imaginary convex shape travel along a line segment before hitting something?
- Two cases:
 1. **Starting in penetration**: Contacts may be inside or on the surface. The shape cannot move.
 2. **Starting free**: The shape moves a nonzero distance. Contacts are always on the surface
- Returns both a **t value** and the **actual contact point**

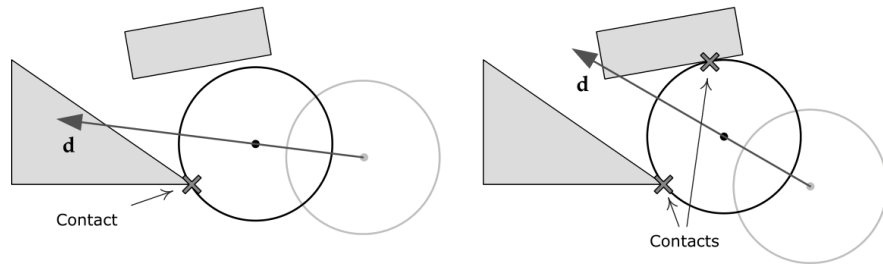


Figure 13.20. If the starting location of a cast shape is not interpenetrating anything, then the shape will move a nonzero distance along its line segment, and its contacts (if any) will always be on its surface.

Shape Cast Contact Structure

- Must always handle **multiple contacts** (even at the earliest t)
- Contact points may not be sorted by t. Sorting manually is almost always a good idea.

```
struct ShapeCastContact
{
    F32 m_t; // the t value for this contact

    U32 m_collidableId; // which collidable did we hit?

    Point m_contactPoint; // location of actual contact

    Vector m_normal; // surface normal at contact pt.

    // other information ...
};
```

Applications of Shape Casts

- **Camera collision**

- Sphere casts to keep the virtual camera from interpenetrating geometry

- **Character movement**

- Cast a sphere/capsule between the character's feet in the direction of motion
- Pop up over low curbs, slide along walls
- Final resting place becomes the character's new location next frame

Phantoms

- Determine which collidables lie within a specific volume
 - (e.g., all enemies within a radius)
- A phantom acts like a shape cast with **distance $d = 0$**
- Unlike a shape cast, a phantom is **persistent** in the collision world
 - Takes full advantage of **temporal coherency** optimizations
- "Invisible" to all other collidables and does not participate in dynamics
- Guarantees no effect on other collidables (including other phantoms)

Collision Filtering

- It is common to want to **enable or disable** collisions between certain object types
- **Collision layers/masking**
 - Each collidable belongs to one collision layer.
 - A bitmask per layer defines which layers can collide
- **Collision callbacks**
 - The collision library invokes a callback when a collision is detected.
 - The callback can accept or reject the contact
- **Collision materials** (game-specific)
 - Associate properties with each collidable surface (sound effects, particle effects, friction coefficients, filtering info)
 - For poly soups, properties may be specified per-triangle

Key Takeaways

- Physics adds realism and interactivity but introduces **unpredictability** and **complexity**.
- Game design should drive physics requirements, not the other way around.
- The collision detection system is fundamentally a **geometric intersection tester** that provides contact information for separation, gameplay events, and physics response.
- Collidable entities use **simple shapes** (spheres, capsules, AABBs, OBBs, convex hulls) and a **transform**, favoring simplicity and efficiency.
- The **separating axis theorem** is a workhorse for intersection testing.
- **GJK** provides a general solution for all convex shape pairs.

Key Takeaways

- Brute force testing is computationally expensive: $O(n^2)$
- Performance depends on:
 - **spatial partitioning** (broad phase, midphase, narrow phase)
 - **temporal coherency**
- Collision queries (ray casts, shape casts, phantoms) are essential tools for line of sight, bullet hits, character movement, and camera collision.

Participation Exercise

Please check Canvas for today's participation exercise.



Questions?